

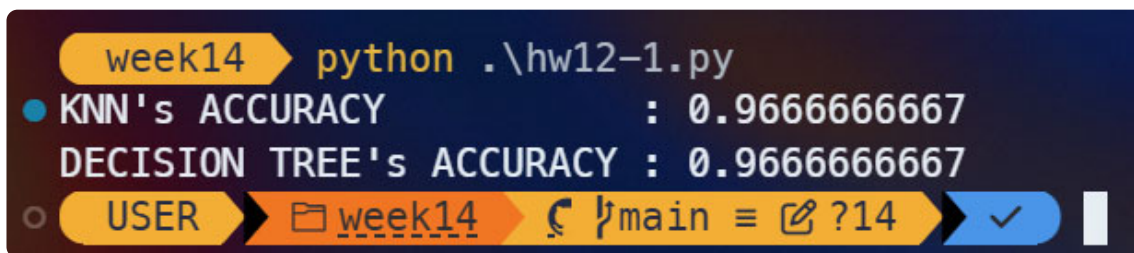
Python Homework 12

Python 程式碼

```
1  import pandas as pd
2  from sklearn.datasets import load_iris
3  from sklearn.preprocessing import StandardScaler
4  from sklearn.model_selection import train_test_split
5  from sklearn.neighbors import KNeighborsClassifier
6  from sklearn.tree import DecisionTreeClassifier
7  from sklearn.metrics import accuracy_score
8
9  iris = load_iris()
10
11 # data preprocessing
12 df_x = pd.DataFrame(
13     data=iris.data,
14     columns=iris.feature_names
15 )
16
17 df_y = pd.Series(
18     data=iris.target,
19     name='target'
20 )
21
22 x_train, x_test, y_train, y_test = train_test_split(
23     df_x,
24     df_y,
25     test_size=0.2,
26     random_state=228922
27 )
28
29 scaler = StandardScaler()
30 x_train_scaled = scaler.fit_transform(x_train)
31 x_test_scaled = scaler.transform(x_test)
32
33 # K-Nearest-Neighbors (KNN)
34 knn = KNeighborsClassifier(n_neighbors=3)
35 knn.fit(x_train_scaled, y_train)
36 knn_predict = knn.predict(x_test_scaled)
37 knn_accuracy = accuracy_score(y_test, knn_predict)
38 print(f"KNN's ACCURACY          : {knn_accuracy:.10f}")
39
40 # Decision Tree
41 dtree = DecisionTreeClassifier(random_state=228922)
42 dtree.fit(x_train_scaled, y_train)
43 dtree_predict = dtree.predict(x_test_scaled)
44 dtree_accuracy = accuracy_score(y_test, dtree_predict)
45 print(f"DECISION TREE's ACCURACY : {knn_accuracy:.10f}")
```

兩種分類方法的準確率結果為何？(並附上輸出結果截圖)

- K-Nearest Neighbors 準確率： $0.9666666667 \approx 96.67\%$
- Decision Tree 準確率： $0.9666666667 \approx 96.67\%$



```
week14 python .\hw12-1.py
• KNN's ACCURACY : 0.9666666667
DECISION TREE's ACCURACY : 0.9666666667
○ USER week14 main ?14 ✓
```

哪一種方法在本資料集上的表現較佳？

在 scikit-learn 套件提供的鸚尾花資料集中，KNN 與決策樹兩種分類方法的準確率皆為 **96.67%**。會造成這個現象的原因，是因為此套件的鸚尾花資料集本身的特徵邊界非常清晰，且較少雜訊干擾。因此無論是基於幾何距離的 KNN 或是基於規則切割的決策樹，都能夠輕鬆地到達非常高的準確率。

兩種分類方法的特性分別為何？

K-Nearest Neighbors (KNN)

- 基於歐式距離的運作邏輯：KNN 並不會事先建立數學模型，而是會將所有訓練資料儲存在記憶體中。當要預測新一筆的資料時，它會先計算該點與所有訓練資料的距離，找出最近的 K 個鄰居，並透過多數決的方式決定新資料的類別。
- 資料處理：由於 KNN 依靠的是計算距離來預測資料，因此這個演算法對數值的範圍非常敏感。若某個特徵的數值範圍很大，它在距離計算中會佔有過大的權重。因此，在執行 KNN 前，必須先將資料集進行標準化 (Standardization)，將所有特徵縮放到相同的尺度。
- 運算特性：
 - 訓練速度極快 → 幾乎不需要運算，只是做資料的儲存。
 - 預測速度稍慢 → 要計算資料與所有點的距離，運算量會隨著資料量的增加而變大。

Decision Tree

- 基於資料規則的運作邏輯：決策樹會透過分析資料特徵，找出能夠區分不同類別的最佳切割點。它會建立一系列的規則，形成一個樹狀結構來進行分類。
- 資料處理：決策樹只關心數值之間的大小關係，而不關心數值的絕對距離。因此，在執行決策樹測前，不用進行標準化等前處理，可以直接將原始數據輸入進行訓練。
- 運算特性：
 - 訓練速度慢 → 需要計算並尋找最佳的特徵切割點來建立樹狀結構。
 - 預測速度快 → 只需順著建立好的規則路徑往下走，運算成本極低。